Exercise Sheet 04

The Roll of the Realm

House rules.

- One file per exercise, in `my_work/`: `ex1_staff_list.py`, `ex2_salaries.py`, …
- Type the **Given** lines first. Then write the rest yourself. Try each step in the REPL before you put it in the file.
- Run it (`python my_work/ex1_staff_list.py`) and compare with the **expected output**, character by character. When Python prints a list or a dictionary itself, it uses single quotes. That's fine; don't fight it.
- Hand in: commit and push to `lesson-04`, then open a pull request. Tell Lars one thing a dictionary does better than a list.

Exercise 1 The Roll

10 min · `ex1_staff_list.py`

> **RAVEN #401, from Mistress Pell, Paymaster:** *"Orrin says you've been keeping the Roll in a text with slashes in it. He said it in the tone he uses for people who store dates as text. Apparently there's a proper thing."*

GIVEN

```
staff = ["Pell Marchbank", "Orrin Ironfoot", "Siv Larkspur", "Bram Kettleby"]
```

(a) Print the first person, the last person, and how many there are.

(b) Elowen Fairweather has joined. `append` her and print the whole list.

(c) Print whether `"Orrin Ironfoot"` is in the list, and whether `"Dalvric"` is.

(d) Bram Kettleby has left. `remove` him and print the list again.

(e) Print a slice of the second and third people. Slicing works like it did on text.

EXPECTED OUTPUT

```
Pell Marchbank
Bram Kettleby
4
['Pell Marchbank', 'Orrin Ironfoot', 'Siv Larkspur', 'Bram Kettleby', 'Elowen Fairweather']
True
False
['Pell Marchbank', 'Orrin Ironfoot', 'Siv Larkspur', 'Elowen Fairweather']
['Orrin Ironfoot', 'Siv Larkspur']
```

Hint: Position 0 is the first, −1 is the last. Same rule as for letters in a string: a list is a row of things, a string is a row of characters.

Exercise 2 A column of numbers

10 min · `ex2_salaries.py`

> **RAVEN #402, from Queen Aldara, at dusk:** *"SUM, MAX, MIN, AVERAGE, and the top three. The things I do in Excel while pretending to listen to the Privy Council. A."*

GIVEN

```
salaries = [31000, 36500, 28000, 24500, 41000, 52000, 19500, 33000]
```

(a) Print the total, then the highest and lowest on one line, then the average rounded to 2 decimals.

(b) Print the salaries sorted low to high, then the top three (sort the other way, slice the first three).

(c) Build a new list `above` of the salaries over 30,000: start with `[]`, loop, `append`. Print it, then "5 of 8 earn over 30,000 crowns" using `len` twice.

EXPECTED OUTPUT

```
265500
52000 19500
33187.5
[19500, 24500, 28000, 31000, 33000, 36500, 41000, 52000]
[52000, 41000, 36500]
[31000, 36500, 41000, 52000, 33000]
5 of 8 earn over 30,000 crowns
```

Hint: `sorted(salaries)` gives a sorted copy and leaves the original alone. `salaries.sort()` sorts in place and returns nothing. Guess which one Dalvric used in a print.

Exercise 3 One subject, many fields

10 min · `ex3_employee_dict.py`

> **RAVEN #403, from Orrin of the Clockworks:** *"A subject isn't a list. A subject has a name, a guild, a salary. In Python that's a dictionary: labels on the left, values on the right. Like one row of a table, with the headers attached."*

GIVEN

```
employee = {
    "name": "Pell Marchbank",
    "department": "Treasury",
    "salary": 31000,
    "hours": 37.5,
}
```

(a) Print the name, then the monthly salary (just divide; no rounding this time).

(b) Add a new field `"rating"` with the value 4, change the salary to 32000, and print the whole dictionary.

(c) Print `employee.get("bonus", 0)`. There is no bonus field: `.get` gives you the fallback instead of crashing. Then print how many fields there are.

(d) Loop over the dictionary (`for key in employee:`) and print each key left-aligned in 12 characters followed by its value.

EXPECTED OUTPUT

```
Pell Marchbank
2583.3333333333335
{'name': 'Pell Marchbank', 'department': 'Treasury', 'salary': 32000, 'hours': 37.5,
'rating': 4}
0
5
name        Pell Marchbank
department  Treasury
salary      32000
hours       37.5
rating      4
```

Exercise 4 A list of dictionaries

15 min · `ex4_staff_records.py`

> **RAVEN #404, from Mistress Pell:** *"So a dictionary is a row and a list is a column. Can I have a table? With people in it? Who each have a salary?"*

GIVEN

```
staff = [
    {"name": "Pell Marchbank", "dept": "Treasury", "salary": 31000, "gender": "F"},
    {"name": "Orrin Ironfoot", "dept": "Clockworks", "salary": 36500, "gender": "M"},
    {"name": "Siv Larkspur", "dept": "Treasury", "salary": 28000, "gender": "F"},
    {"name": "Bram Kettleby", "dept": "Harbour", "salary": 24500, "gender": "M"},
    {"name": "Elowen Fairweather", "dept": "Guard", "salary": 41000, "gender": "F"},
]
```

(a) Loop over the list: each `person` is a dictionary. Print one line each: name in 20, guild in 12, then the salary as `31,000 cr` right-aligned in 9. (Build the money text first, then align it.)

(b) Add up the salaries into a total and print it.

(c) Find the highest-paid person. Start with `top = staff[0]`, loop, and replace `top` whenever someone earns more. Print their name and guild.

(d) Build a list of the women's names and print it.

EXPECTED OUTPUT

```
Pell Marchbank      Treasury    31,000 cr
Orrin Ironfoot      Clockworks  36,500 cr
Siv Larkspur        Treasury    28,000 cr
Bram Kettleby       Harbour     24,500 cr
Elowen Fairweather  Guard       41,000 cr
Total payroll: 161,000 cr
Highest paid: Elowen Fairweather (Guard)
['Pell Marchbank', 'Siv Larkspur', 'Elowen Fairweather']
```

Hint: Inside an f-string, use the other kind of quotes for the key: `f"{person['name']}"`.

Exercise 5 Counting with a dictionary

10 min · `ex5_headcount.py`

> **RAVEN #405, from Lady Siv of the Rewards:** *"Headcount per guild. In Excel it's a COUNTIF per guild, and someone always forgets the new guild. I want it to notice new guilds by itself."*

GIVEN

```
departments = ["Guard", "Clockworks", "Treasury", "Guard", "Harbour", "Clockworks", "Guard",
"Kitchens", "Clockworks", "Guard"]
```

(a) Start with an empty dictionary `headcount = {}`. Loop over the guilds: if the guild is already a key, add 1; otherwise create it with 1. Print the dictionary.

(b) Print the counts as a table, guilds in alphabetical order (11 characters for the name) (`sorted(headcount)` sorts the keys).

(c) Find the biggest guild and print it with its count.

EXPECTED OUTPUT

```
{'Guard': 4, 'Clockworks': 3, 'Treasury': 1, 'Harbour': 1, 'Kitchens': 1}
Clockworks   3
Guard        4
Harbour      1
Kitchens     1
```

```
Treasury     1
Biggest guild: Guard (4 people)
```

Hint: This "if it's there add, otherwise create" loop is the most common five lines in all of data work. You'll write it a hundred times, until pandas does it for you in one.

Exercise 6 XLOOKUP, the Python way

10 min · `ex6_lookup.py`

> **RAVEN #406, from Mistress Pell:** *"Half my spreadsheets are XLOOKUPs: band to minimum salary, guild to guildmaster. Tell me a dictionary is an XLOOKUP and I'll stop being scared of it."*

GIVEN

```
band_minimum = {"A": 0, "B": 25000, "C": 32000, "D": 40000}
dept_manager = {"Guard": "Elowen Fairweather", "Clockworks": "Orrin Ironfoot", "Treasury":
"Pell Marchbank"}
```

(a) Look up band C's minimum and the Clockworks' guildmaster. Then look up the Kitchens' guildmaster with `.get` and the fallback `"no guildmaster yet"`. Then print whether `"Treasury"` is a key.

(b) Print the keys of `dept_manager` as a list, and the values of `band_minimum` as a list.

(c) Loop with `for band, minimum in band_minimum.items():` and print one line per band.

(d) Given `salary = 33000`, work out its band: loop over the items, and whenever the salary is at least the minimum, remember that band. Print the result.

EXPECTED OUTPUT

```
32000
Orrin Ironfoot
no guildmaster yet
True
['Guard', 'Clockworks', 'Treasury']
[0, 25000, 32000, 40000]
Band A starts at 0 cr
Band B starts at 25,000 cr
Band C starts at 32,000 cr
Band D starts at 40,000 cr
33,000 cr is in band C
```

Exercise 7 The pay gap, by guild STRETCH

15 min · `ex7_pay_gap_by_dept.py`

> **RAVEN #407, from Queen Aldara:** *"Pay gap per guild. For the Privy Council. By Thursday. (Which Thursday? A Thursday.) A."*

GIVEN

```
staff = [
    {"name": "Pell Marchbank", "dept": "Treasury", "hourly": 15.90, "gender": "F"},
    {"name": "Orrin Ironfoot", "dept": "Clockworks", "hourly": 18.72, "gender": "M"},
    {"name": "Siv Larkspur", "dept": "Treasury", "hourly": 14.36, "gender": "F"},
    {"name": "Bram Kettleby", "dept": "Clockworks", "hourly": 12.56, "gender": "M"},
    {"name": "Elowen Fairweather", "dept": "Clockworks", "hourly": 21.03, "gender": "F"},
    {"name": "Emrys Oakhollow", "dept": "Treasury", "hourly": 17.20, "gender": "M"},
    {"name": "Wren Thistlewood", "dept": "Clockworks", "hourly": 16.41, "gender": "F"},
]
```

(a) Write two functions: `average(numbers)` and `pay_gap_percent(male_hourly, female_hourly)` (as % of men's rate, 1 decimal).

(b) Build a dictionary `rates` whose keys look like `"Clockworks/F"` and whose values are **lists** of hourly rates. The counting loop from exercise 5, but appending instead of adding 1. Print it.

(c) For `"Treasury"` and `"Clockworks"`, print the men's and women's average and the gap.

EXPECTED OUTPUT

```
{'Treasury/F': [15.9, 14.36], 'Clockworks/M': [18.72, 12.56], 'Clockworks/F': [21.03,
16.41], 'Treasury/M': [17.2]}
Treasury    men 17.20 cr  women 15.13 cr  gap 12.0%
Clockworks  men 15.64 cr  women 18.72 cr  gap -19.7%
```

Hint: A dictionary whose values are lists is a dictionary like any other. `rates[key].append(x)` reads as: find the list for this key, then append to it.

Bug hunt Pages from the Grimoire, page 4

15 min · `bughunt/staff_list_bug.py`

Dalvric's Roll of the Realm, last run the week he sailed west. It found the wrong top earner, crashed looking up a guild that definitely exists, and reported a headcount of 1 for everyone. There are **three** bugs: one crashes (read the error twice; the clue is in the quotes), one picks the wrong top earner silently, and one makes every count come out as 1.

EXPECTED OUTPUT ONCE FIXED

```
Highest paid: Elowen Fairweather
Clockworks: Orrin Ironfoot
{'Treasury': 1, 'Clockworks': 2, 'Guard': 1, 'Harbour': 1}
```

Commit each fix separately, with a message explaining what was wrong, like a detective's report.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.